

# **Software Architecture**

---

**Presented by:** Dr. Abdolraheem Khader  
May 2025

# Introduction to Software Architecture

---

- **Definition:** High-level structure of a software system, defining its components and interactions.
- **Objective:** Provide a blueprint for system construction and maintenance.
- **Importance:** Facilitates scalability, maintenance, and collaboration.



# Core Aspects of Software Architecture

---

- **Components:** Individual parts that perform specific functions.
- **Connectors:** Communication mechanisms between components.
- **Configurations:** The arrangement of components and connectors.
- **Styles:** Design patterns used to structure the architecture (e.g., MVC).

# Part 1: Layered Architecture

---

- **Definition:** Divides the system into layers, each with a specific responsibility.
- **Common Layers:**
  - Presentation Layer.
  - Application Layer.
  - Business Logic Layer.
  - Data Access Layer.
- **Benefits:**
  - Separation of concerns, Ease of maintenance, and Scalability.



# Part 1: Components of Layered Architecture

---

- **Presentation Layer:** User interface and experience.
- **Business Logic Layer:** Core functionality and rules.
- **Data Access Layer:** Interaction with databases.
- **Database Layer:** Data storage and management.
- **Example:** A web application with a frontend, backend, and database.

# Part 1: Layered Architecture: Pros and Cons

---

- **Pros:**

- Clear separation of concerns.
- Easier testing and maintenance.
- Enhanced reusability.

- **Cons:**

- Performance overhead due to multiple layers.
- Rigidity when adding new features.

- **Use Case:** Enterprise applications.



# Layered Architecture Example: E-Commerce System

---

- **Presentation Layer:** Web and mobile interfaces.
- **Business Logic Layer:** Order processing and inventory management.
- **Data Access Layer:** SQL queries and data manipulation.
- **Database Layer:** MySQL or MongoDB.

# Implementation Guidelines

---

- **Modularity:** Keep each layer self-contained.
- **Independence:** Avoid direct communication between non-adjacent layers.
- **Data Flow:** Unidirectional from presentation to database and back.
- **Example:** Using APIs to separate frontend from backend logic.



## Part 2: Client-Server Architecture

---

- **Definition:** A network architecture where clients request services from a centralized server.
- **Components:**
  - Client: Initiates requests.
  - Server: Responds to client requests.
- **Example:** Web browsers accessing a web server.

## Part 2: Characteristics of Client-Server Architecture

---

- **Centralization:** Servers handle most processing tasks.
- **Scalability:** Adding more clients without changing the server.
- **Communication:** Through protocols (e.g., HTTP, FTP)
- **Example:** Online banking applications



# Part 2: Client-Server Architecture: Pros and Cons

---

- **Pros:**

- Centralized data management.
- Easy maintenance.
- Load distribution across clients.

- **Cons:**

- Server overload risks.
- Dependency on server availability.

- **Use Case:** Email systems.

# Real-World Example: Social Media Platform

---

- **Client:** Mobile app or web browser.
- **Server:** API server for fetching posts and user data.
- **Communication:** HTTP requests for data retrieval and updates.



# Client-Server Interaction Flow

---

1. Client sends a request (HTTP GET/POST).
  2. Server processes the request.
  3. Server sends a response (data or error message).
  4. Client displays the data.
- **Example:** User logs in to an e-commerce website

# Part 3: Microservices Architecture

---

- **Definition:** Breaks down a system into small, loosely coupled services.
- **Characteristics:**
  - Independent deployment.
  - Fine-grained scalability.
  - Decentralized data management.
- **Example:** Netflix microservices architecture.



# Key Components of Microservices

---

- **Services:** Independently deployable units.
- **API Gateway:** Centralized entry point.
- **Service Registry:** Tracks available services.
- **Message Broker:** Enables communication.
- **Example:** Payment, user, and product services in an e-commerce app.

# Microservices: Pros and Cons

---

- **Pros:**

- Scalability and flexibility.
- Technology diversity.
- Fault isolation.

- **Cons:**

- Increased complexity.
- Challenging data consistency.
- Deployment overhead.

- **Use Case:** Large-scale, dynamic applications.



# Real-World Example: Uber

---

- **Services:**
  - User management.
  - Ride matching.
  - Payment processing.
- **Benefits:**
  - Independent updates.
  - High availability.
  - Load balancing.

# Part 4: MVC Architecture (Model-View-Controller)

---

- **Definition:** A design pattern that separates application logic into three interconnected components.
- **Components:**
  - Model: Data and business logic.
  - View: User interface.
  - Controller: Intermediary between Model and View.
- **Example:** Web application frameworks like Django and Ruby on Rails.



# Model-View-Controller Flow

---

1. User interacts with the View.
  2. Controller processes input and updates the Model.
  3. Model notifies the View to update.
  4. View displays updated data.
- **Example:** User submitting a form on a website.

# MVC: Pros and Cons

---

- **Pros:**

- Separation of concerns.
- Reusable components.
- Enhanced testing.

- **Cons:**

- Complexity in small applications.
- Tight coupling between Controller and View.

- **Use Case:** Web applications.



# MVC Example: Blog Platform

---

- **Model:** Post, Comment, User.
- **View:** HTML pages displaying posts.
- **Controller:** Handling user requests and updating the Model.

# Comparative Analysis

---

- **Layered vs. Client-Server Architecture:**
  - Layered: Modular, suitable for large applications.
  - Client-Server: Centralized, better for networked systems.
- **Microservices vs. MVC:**
  - Microservices: Loosely coupled, scalable.
  - MVC: Structured, effective for UI-centric apps.



# Case Studies and Real-World Applications

---

- **E-Commerce System Architecture:**
  - **Hybrid Approach:** Layered for internal processing, MVC for user interactions, microservices for scalability.
- **Social Media Platform:**
  - **Architecture:** Client-Server for user interaction, Microservices for data management.

# Challenges and Best Practices

---

- **Challenges in Software Architecture:**
  - Balancing performance and maintainability.
  - Handling complexity in microservices.
  - Maintaining consistency in distributed systems.
- **Best Practices:**
  - Follow the **Separation of Concerns** principle.
  - Utilize **APIs** for modularity.
  - Ensure **fault tolerance** in microservices.



# Future Trends in Software Architecture

---

- **Serverless Architecture:** Event-driven, no dedicated servers.
- **Event-Driven Microservices:** Asynchronous processing.
- **Edge Computing:** Processing closer to the data source.

---

Thank You!